



操作系统第四次实验

姓名： 江李阳 王健一 邱泽辉

学号： 202400460042
202400460037
202400460046

专业： 密码科学与技术

日期： 2026 年 5 月 31 日

目录

1 实验目的	2
2 实验环境	2
3 实验原理	2
3.1 FAT	2
4 实验概述	2
4.1 任务一	2
4.2 任务二	2
5 任务一	3
5.1 任务要求	3
5.2 实验过程	3
5.2.1 磁盘空间划分与整体设计	3
5.2.2 文件系统基本数据结构设计	4
5.2.3 文件系统初始化	5
5.2.4 数据块分配与释放	6
5.2.5 目录项查找与目录创建	7
5.2.6 文件打开与关闭	8
5.2.7 文件写入与读取	9
5.2.8 文件删除与空间回收	10
5.2.9 跨块文件写入测试	11
5.2.10 磁盘空间划分	13
5.2.11 FAT 表设计	13
5.2.12 目录项设计	14
5.2.13 基本文件操作实现	15
5.2.14 运行结果与分析	16

1 实验目的

- 任务一: 实现一个简单的类 FAT-16 文件系统。
- 任务二: 实现多个进程同时访问文件、目录的功能。

2 实验环境

3 实验原理

3.1 FAT

显式链接是文件分配物理块的一种方式 (类似静态链表), 其中将链接文件所有块的指针显示地存入内存的一张表中, 即文件分配表 (FAT, File Allocation Table, 每个表项存放链接指针, 即下一盘块号。因此只要将文件的第一个盘块号存放在 FCB 中, 后续盘块号都可以通过查 FAT 表来获取。因此 FAT 的表项与全部磁盘块一一对应, 可以使用一个特殊数字如-1 表示空闲块,-2 表示文件结束块。也就是说 FAT 不仅记录了文件各块之间的前后链接关系, 还标记了空闲块信息, 还可以用来实现文件管理如分配空闲磁盘块等。FAT 的优点是该表在系统系统时读入内存, 因此查找记录的过程是在内存中进行的, 因此检索速度很快, 减少了访问磁盘的次数。缺点是每次都要将整个 FAT 读入内存, 这会占用较大的内存空间。

4 实验概述

本次实验围绕类 FAT-16 文件系统的设计与并发访问控制展开, 主要包括两个任务。任务一要求在内存中构造一个简化的文件系统, 将数组 `Disk[DISK_MAXLEN]` 模拟为硬盘空间, 并在此基础上实现目录创建、目录查看、文件删除、文件打开、文件读取、文件写入和文件关闭等基本文件系统操作。任务二则在任务一的基础上进一步考虑多个进程同时访问文件和目录时的同步问题, 通过信号量机制保证文件系统在并发访问过程中的正确性。

4.1 任务一

在任务一中, 实验的重点是完成文件系统基本结构的设计, 包括磁盘空间划分、FAT 表维护、目录项组织以及文件数据块管理。通过将内存数组划分为超级块、FAT 表、根目录区和数据区, 可以较为清晰地模拟真实文件系统中文件和目录的存储方式。其中, FAT 表用于记录数据块之间的链接关系, 目录项用于保存文件名、文件类型、起始块号和文件大小等元信息。通过这些结构, 可以实现文件创建、写入、读取和删除等操作, 并观察文件内容在模拟磁盘中的组织过程。

4.2 任务二

在任务二中, 实验进一步引入多进程访问场景。由于多个进程可能同时访问同一个文件或目录, 因此需要通过信号量对共享资源进行保护。对于目录项、FAT 表和磁盘数据区等全局共享结构, 需要使用互斥锁避免多个进程同时修改造成数据不一致; 对于普通文件的访问, 则需要满足一个文件可以被多个进程同时读取, 但同一时刻只能被一个进程写入的要求 (类似于之前的读者-写者问题)。

因此，本任务的关键在于设计合理的读写同步机制，使文件系统既能保证并发访问的安全性，又能允许多个读进程并发执行，提高系统资源利用率。

5 任务一

5.1 任务要求

本任务要求实现一个简单的类 FAT-16 文件系统，并支持目录创建、目录查看、文件删除、文件打开、文件读取、文件写入和文件关闭等基本功能。实验中不使用真实磁盘，而是通过定义内存数组：

```
1 #define DISK_MAXLEN 2560
2 char Disk[DISK_MAXLEN];
```

将该数组模拟为一块硬盘空间，在此基础上完成文件系统的数据组织和文件操作。

根据实验要求，本文件系统只需要考虑单进程访问场景，因此暂不需要处理多个进程同时访问文件或目录时的同步问题。实验重点在于设计并实现文件系统的核心数据结构，包括超级块、FAT 表、目录项和数据块等内容。其中，FAT 表用于维护文件所占数据块之间的链接关系，目录项用于记录文件名、文件类型、起始块号和文件大小等元信息，数据块则用于保存具体的文件或目录内容。

通过本任务，需要完成以下基本操作：

- **mkdir**: 在当前目录下创建新的子目录；
- **ls**: 显示当前目录下的文件和子目录信息；
- **delete**: 删除指定文件或空目录，并释放其占用的数据块；
- **open**: 打开指定文件，返回文件描述符；
- **read**: 根据文件描述符读取文件内容；
- **write**: 向指定文件写入数据，并更新 FAT 表和目录项；
- **close**: 关闭已经打开的文件。

因此，本任务的核心目标是通过内存模拟的方式，完成一个具备基本目录管理、文件读写和空间回收能力的简化文件系统，并通过运行结果验证各项文件操作的正确性。

5.2 实验过程

5.2.1 磁盘空间划分与整体设计

本任务使用内存数组 `Disk[DISK_MAXLEN]` 模拟硬盘空间。由于实验中定义的磁盘总大小为 2560 字节，因此本实验将磁盘划分为若干个固定大小的数据块，每个数据块大小设置为 128 字节，总共可以划分为 20 个块。

```
1 #define DISK_MAXLEN 2560
2 #define BLOCK_SIZE 128
```

```
3 #define BLOCK_NUM (DISK_MAXLEN / BLOCK_SIZE)
```

在此基础上，文件系统的磁盘布局设计如下：

- 第 0 块：超级块，用于保存文件系统的基本信息；
- 第 1 块：FAT 表，用于记录各数据块的使用状态和链接关系；
- 第 2 块：根目录区，用于保存根目录下的目录项；
- 第 3 块到第 19 块：数据区，用于保存文件或子目录内容。

其中，FAT 表是本实验实现类 FAT-16 文件系统的核心结构。每个 FAT 表项对应一个数据块，其取值含义如下：

- FAT_FREE：表示该块空闲；
- FAT_END：表示该块已经被占用，并且是文件或目录的最后一个块；
- 其他块号：表示当前块的下一个数据块编号。

因此，当一个文件只占用一个数据块时，对应 FAT 表项为 FAT_END；当一个文件跨越多个数据块时，FAT 表会形成一条链式结构，例如 4 -> 5 -> 6 -> END。

5.2.2 文件系统基本数据结构设计

为了保存文件系统的基本信息，本实验定义了超级块结构体，用于记录块大小、块数量、FAT 表位置、根目录位置和数据区起始位置等信息。

```
1 typedef struct {
2     int block_size;
3     int block_num;
4     int fat_start;
5     int root_dir_block;
6     int data_start;
7 } SuperBlock;
```

同时，为了描述文件和目录，本实验定义了目录项结构体。每个目录项记录一个文件或子目录的基本信息，包括名称、是否使用、类型、起始块号和文件大小。

```
1 typedef struct {
2     char name[NAME_LEN];
3     unsigned char used;
4     unsigned char type;
5     unsigned short first_block;
6     unsigned short size;
7     char reserved[10];
8 } DirEntry;
```

其中，type 字段用于区分普通文件和目录，first_block 字段表示文件或目录内容在数据区中的起始块号，size 字段用于记录文件大小。通过目录项和 FAT 表的配合，系统可以根据文件名找到其起始块，并沿 FAT 链读取或释放文件内容。

5.2.3 文件系统初始化

文件系统初始化由 `fs_format()` 函数完成。该函数首先清空整个 `Disk` 数组，然后初始化超级块信息，并将 FAT 表全部设置为空闲状态。随后，将第 0 块、第 1 块和第 2 块分别标记为已占用，因为它们分别用于超级块、FAT 表和根目录。

初始化后的 FAT 表状态如下：

- Block 0: END，表示超级块已占用；
- Block 1: END，表示 FAT 表已占用；
- Block 2: END，表示根目录已占用；
- Block 3 到 Block 19 均为 FREE，表示数据区尚未分配。

运行结果中，初始目录为空，FAT 表中只有前三个块被占用，其余块均为空闲状态，说明文件系统格式化过程正确。

```
format success

[1] initial ls
===== ls block 2 =====
=====
===== FAT Table =====
Block 0: END
Block 1: END
Block 2: END
Block 3: FREE
Block 4: FREE
Block 5: FREE
Block 6: FREE
Block 7: FREE
Block 8: FREE
Block 9: FREE
Block 10: FREE
Block 11: FREE
Block 12: FREE
Block 13: FREE
Block 14: FREE
Block 15: FREE
Block 16: FREE
Block 17: FREE
Block 18: FREE
Block 19: FREE
=====
```

图 1: 文件系统初始化与 FAT 表初始状态

5.2.4 数据块分配与释放

为了支持文件和目录的创建，本实验实现了数据块分配函数 `alloc_block()`。该函数从数据区起始块开始扫描 FAT 表，寻找空闲块。如果找到空闲块，则将其 FAT 表项设置为 `FAT_END`，并清

空该块内容，最后返回该块编号。

文件删除或覆盖写入时，需要释放原文件占用的数据块。因此，本实验实现了 `free_chain()` 函数。该函数从文件起始块开始，沿 FAT 表记录的链式结构逐块向后遍历，并将经过的块重新设置为 `FAT_FREE`，从而完成空间回收。

这两个函数是后续 `mkdir`、`write` 和 `delete` 操作的基础。其中，`alloc_block()` 负责从空闲空间中分配数据块，`free_chain()` 负责在删除文件时回收文件占用的全部数据块。

5.2.5 目录项查找与目录创建

为了在目录中查找文件或子目录，本实验实现了目录项查找函数 `find_entry()` 和空目录项查找函数 `find_empty_entry()`。其中，`find_entry()` 根据文件名在当前目录中查找已有目录项，`find_empty_entry()` 用于寻找尚未使用的目录项位置。

在实现 `mkdir` 操作时，系统首先检查当前目录下是否已经存在同名目录或文件。如果不存在，则调用 `alloc_block()` 分配一个新的数据块作为子目录的数据块，然后在当前目录中写入新的目录项。目录项中记录目录名、目录类型、起始块号和大小等信息。

实验中执行：

```
1 fs_mkdir(ROOT_BLOCK, "dir1");
```

运行结果显示：

```
1 mkdir success: dir1, block=3
2 [DIR ] dir1      first=3 size=0
```

同时，FAT 表中 Block 3 由 `FREE` 变为 `END`。这说明系统成功为目录 `dir1` 分配了第 3 块，并在根目录中建立了对应目录项。


```
[2] mkdir dir1
mkdir success: dir1, block=3
===== ls block 2 =====
[DIR ] dir1      first=3 size=0
=====
===== FAT Table =====
Block 0: END
Block 1: END
Block 2: END
Block 3: END
Block 4: FREE
Block 5: FREE
Block 6: FREE
Block 7: FREE
Block 8: FREE
Block 9: FREE
Block 10: FREE
Block 11: FREE
Block 12: FREE
Block 13: FREE
Block 14: FREE
Block 15: FREE
Block 16: FREE
Block 17: FREE
Block 18: FREE
Block 19: FREE
=====
```

图 2: 创建目录 dir1 后的目录项与 FAT 表变化

5.2.6 文件打开与关闭

为了管理已经打开的文件，本实验设计了一个简单的打开文件表 `open_table`。每个打开文件表项记录文件是否被占用、所在目录块、文件名和打开模式等信息。

```
1 typedef struct {
2     int used;
3     int dir_block;
4     char filename[NAME_LEN];
5     char mode;
6 } OpenFile;
```

当执行 `open` 操作时，系统根据文件名在当前目录中查找文件。如果以读模式打开，则文件必须已经存在；如果以写模式打开，则当文件不存在时，系统会自动创建新的文件目录项。随后，系统在打开文件表中寻找空闲表项，并返回对应的文件描述符。

当执行 `close` 操作时，系统只需要将打开文件表中对应表项的 `used` 字段清零，表示该文件描述符已经释放。

实验中，文件 `a.txt` 被成功打开，并返回文件描述符 `fd=0`，说明打开文件表能够正常管理文件打开状态。

5.2.7 文件写入与读取

文件写入由 `fs_write()` 实现。本实验采用覆盖写的方式，即每次写入文件时，如果文件原来已经占用数据块，则先调用 `free_chain()` 释放原有 FAT 链，然后根据新写入内容的长度重新分配数据块，并将这些数据块通过 FAT 表连接起来。

当写入内容长度小于一个块大小时，文件只需要占用一个数据块。例如实验中向 `a.txt` 写入字符串：

```
1 fs_write(fd, "Hello FAT File System!");
```

运行结果显示：

```
1 [FILE] a.txt      first=4 size=22
2 Block 4: END
```

这说明 `a.txt` 的起始块为第 4 块，文件大小为 22 字节。由于该文件内容小于一个块大小，所以只占用一个数据块，FAT 表中 Block 4 被标记为 END。

文件读取由 `fs_read()` 实现。读取时，系统首先根据文件描述符找到对应目录项，再根据目录项中的 `first_block` 找到文件起始块，并沿 FAT 链依次读取文件内容。实验运行结果显示：

```
1 read result: Hello FAT File System!
```

这说明系统能够根据目录项和 FAT 表正确定位文件数据，并成功读取文件内容。

```
[3] open and write a.txt
open success: a.txt, fd=0, mode=w
write success: fd=0, size=22 bytes
close success: fd=0
===== ls block 2 =====
[DIR ] dir1      first=3 size=0
[FILE] a.txt     first=4 size=22
=====
===== FAT Table =====
Block 0: END
Block 1: END
Block 2: END
Block 3: END
Block 4: END
Block 5: FREE
Block 6: FREE
Block 7: FREE
Block 8: FREE
Block 9: FREE
Block 10: FREE
Block 11: FREE
Block 12: FREE
Block 13: FREE
Block 14: FREE
Block 15: FREE
Block 16: FREE
Block 17: FREE
Block 18: FREE
Block 19: FREE
=====

[4] read a.txt
open success: a.txt, fd=0, mode=r
read success: fd=0, size=22 bytes
read result: Hello FAT File System!
close success: fd=0
```

图 3: 文件 a.txt 的写入与读取结果

5.2.8 文件删除与空间回收

文件删除由 `fs_delete()` 实现。删除普通文件时，系统首先在当前目录中查找目标文件目录项。如果文件存在，则根据目录项中的起始块号调用 `free_chain()` 释放其占用的所有数据块，最后清空对应目录项。

实验中删除 `a.txt` 后，运行结果显示：

```
1 delete file success: a.txt
2 Block 4: FREE
```

这说明文件删除后，`a.txt` 原本占用的第 4 块已经被重新标记为空闲状态，文件系统完成了空间回收。该结果表明，本实验实现的 `delete` 操作不仅删除了目录项，而且正确释放了文件数据块。

```
[5] delete a.txt
delete file success: a.txt
===== ls block 2 =====
[DIR ] dir1          first=3 size=0
=====
===== FAT Table =====
Block  0: END
Block  1: END
Block  2: END
Block  3: END
Block  4: FREE
Block  5: FREE
Block  6: FREE
Block  7: FREE
Block  8: FREE
Block  9: FREE
Block 10: FREE
Block 11: FREE
Block 12: FREE
Block 13: FREE
Block 14: FREE
Block 15: FREE
Block 16: FREE
Block 17: FREE
Block 18: FREE
Block 19: FREE
=====
```

图 4: 删除 `a.txt` 后的目录项与 FAT 表变化

5.2.9 跨块文件写入测试

为了进一步验证 FAT 表的链式存储机制，本实验额外写入了一个长度为 299 字节的文件 `big.txt`。由于每个数据块大小为 128 字节，因此该文件无法存放在单个数据块中，需要连续分配多个数据块，

并通过 FAT 表建立块之间的链接关系。

实验运行结果显示：

```
1 [FILE] big.txt      first=4 size=299
2 Block 4: 5
3 Block 5: 6
4 Block 6: END
```

该结果说明，big.txt 的起始块为第 4 块，并依次占用了第 4、5、6 三个数据块。其中，FAT 表项 Block 4: 5 表示第 4 块之后连接第 5 块，Block 5: 6 表示第 5 块之后连接第 6 块，Block 6: END 表示第 6 块是文件的最后一个数据块。

因此，big.txt 在文件系统中的存储结构可以表示为：

Block 4 -> Block 5 -> Block 6 -> END

随后，系统对 big.txt 进行读取，能够正确输出文件前 100 个字符，说明跨块文件不仅能够正确写入，而且能够沿 FAT 链正确读取。这一结果较好地验证了类 FAT-16 文件系统中链式块管理机制的正确性。

```
[6] write long file big.txt
open success: big.txt, fd=0, mode=w
write success: fd=0, size=299 bytes
close success: fd=0
===== ls block 2 =====
[DIR ] dir1      first=3 size=0
[FILE] big.txt   first=4 size=299
=====
===== FAT Table =====
Block 0: END
Block 1: END
Block 2: END
Block 3: END
Block 4: 5
Block 5: 6
Block 6: END
Block 7: FREE
Block 8: FREE
Block 9: FREE
Block 10: FREE
Block 11: FREE
Block 12: FREE
Block 13: FREE
Block 14: FREE
Block 15: FREE
Block 16: FREE
Block 17: FREE
Block 18: FREE
Block 19: FREE
=====

[7] read long file big.txt
open success: big.txt, fd=0, mode=r
read success: fd=0, size=299 bytes
read big.txt first part:
ABCDEFGHIJKLMNQRSTUvwXYZABCDEFGHIJKLMNQRSTUvwXYZABCDEFGHIJKLMNQRSTUv
close success: fd=0
```

图 5: big.txt 跨块写入后的 FAT 链式结构

“latex

5.2.10 磁盘空间划分

本实验使用内存数组 `Disk` 模拟一块容量为 2560 字节的硬盘，并以固定大小的数据块作为磁盘空间分配的基本单位。相关宏定义如下：

```
1 #define DISK_MAXLEN 2560
2 #define BLOCK_SIZE 128
3 #define BLOCK_NUM (DISK_MAXLEN / BLOCK_SIZE)
4
5 char Disk[DISK_MAXLEN];
```

由于每个数据块大小为 128 字节，因此整个模拟磁盘可以划分为 20 个块。为了分别保存文件系统元数据、目录信息和文件内容，本实验将磁盘空间划分为以下几个区域：

- 第 0 块：超级块，用于保存文件系统的基本配置信息；
- 第 1 块：FAT 表，用于记录各数据块的使用状态和链接关系；
- 第 2 块：根目录区，用于保存根目录下的目录项；
- 第 3 块至第 19 块：数据区，用于保存普通文件内容或子目录内容。

为了记录文件系统的整体布局，本实验定义了超级块结构体：

```
1 typedef struct {
2     int block_size;
3     int block_num;
4     int fat_start;
5     int root_dir_block;
6     int data_start;
7 } SuperBlock;
```

其中，`block_size` 和 `block_num` 分别记录数据块大小和数据块总数，`fat_start` 表示 FAT 表所在位置，`root_dir_block` 表示根目录所在块，`data_start` 表示数据区的起始块号。

在文件系统初始化过程中，第 0、1、2 块会被标记为已经占用，其余数据块均处于空闲状态。这样的划分方式虽然比真实 FAT-16 文件系统更加简化，但已经能够支持本实验要求的目录管理、文件读写和空间回收功能。

5.2.11 FAT 表设计

FAT 表是本实验实现类 FAT-16 文件系统的核心数据结构。每个 FAT 表项与一个磁盘块一一对应，用于记录该数据块当前是否空闲，以及该数据块之后是否还连接其他数据块。

本实验使用 `unsigned short` 类型保存 FAT 表项，并定义了两种特殊状态：

```
1 #define FAT_FREE 0x0000
2 #define FAT_END 0xFFFF
```

不同 FAT 表项取值的含义如下：

- `FAT_FREE`：表示当前数据块尚未分配，可以被新文件或目录使用；

- **FAT_END**: 表示当前数据块已经被占用, 并且是文件或目录占用的最后一个块;
- 其他块号: 表示当前数据块之后连接的下一个数据块编号。

例如, 当一个文件只占用第 4 块时, 其 FAT 表项为:

Block 4: END

当一个文件占用多个数据块时, FAT 表会形成链式结构。例如, 一个文件依次占用第 4、5、6 块, 则 FAT 表中的记录为:

Block 4: 5

Block 5: 6

Block 6: END

对应的数据块链接关系可以表示为:

Block 4 → Block 5 → Block 6 → END

为了支持数据块的动态管理, 本实验实现了 `alloc_block()` 和 `free_chain()` 两个辅助函数。

`alloc_block()` 从第 3 块开始扫描 FAT 表, 寻找状态为 `FAT_FREE` 的数据块。找到空闲块后, 将其标记为 `FAT_END`, 清空对应块中的数据, 并返回该块编号。

`free_chain()` 用于释放文件占用的数据块。该函数从文件起始块开始, 根据 FAT 表项逐块向后遍历, 将经过的所有数据块重新设置为 `FAT_FREE`, 直到遇到 `FAT_END` 为止。该函数主要用于文件删除和覆盖写入操作。

通过上述设计, 文件的数据块不需要在磁盘中连续分布, 只需要由 FAT 表维护块之间的链接关系即可。

5.2.12 目录项设计

目录用于保存文件和子目录的基本信息。本实验将目录同样视为一种特殊的数据内容, 每个目录占用一个数据块, 目录块中保存若干个目录项。

目录项结构体定义如下:

```
1 typedef struct {
2     char name[NAME_LEN];
3     unsigned char used;
4     unsigned char type;
5     unsigned short first_block;
6     unsigned short size;
7     char reserved[10];
8 } DirEntry;
```

各字段含义如下:

- **name**: 记录文件名或目录名;
- **used**: 记录当前目录项是否已经被使用;

- **type:** 区分普通文件和目录;
- **first_block:** 记录文件或子目录内容所在的起始块号;
- **size:** 记录普通文件的大小;
- **reserved:** 预留字段, 用于保证目录项结构具有较为规整的大小。

为了区分普通文件和目录, 本实验定义了两种目录项类型:

```
1 #define TYPE_FILE 1
2 #define TYPE_DIR 2
```

例如, 创建目录 `dir1` 后, 根目录中会新增如下目录项:

```
[DIR ] dir1  first=3  size=0
```

其中, `first=3` 表示目录 `dir1` 占用第 3 块, `size=0` 表示该目录初始为空。

创建文件 `a.txt` 并写入数据后, 根目录中会新增如下目录项:

```
[FILE] a.txt  first=4  size=22
```

其中, `first=4` 表示文件内容从第 4 块开始存储, `size=22` 表示文件大小为 22 字节。

为了实现目录项管理, 本实验还实现了两个辅助函数:

- `find_entry()`: 根据文件名或目录名, 在当前目录中查找对应目录项;
- `find_empty_entry()`: 在当前目录中寻找尚未使用的目录项, 用于创建新文件或新目录。

本实验采用简化设计, 每个目录占用一个固定大小的数据块。该设计能够满足本次实验对目录创建、目录查看和文件管理的基本要求。

5.2.13 基本文件操作实现

在完成磁盘划分、FAT 表和目录项设计后, 本实验实现了 `mkdir`、`ls`、`delete`、`open`、`read`、`write` 和 `close` 等基本文件操作。

1. 创建目录 目录创建功能由 `fs_mkdir()` 实现。该函数首先检查当前目录中是否已经存在同名文件或目录。如果不存在, 则调用 `alloc_block()` 分配新的数据块, 并在当前目录中寻找空闲目录项。随后, 将目录名、目录类型和起始块号写入目录项。

其基本流程如下:

1. 检查当前目录中是否已经存在同名目录项;
2. 在当前目录中寻找空闲目录项;
3. 从 FAT 表中分配一个空闲数据块;
4. 初始化新目录对应的数据块;
5. 在当前目录中写入新目录项。

2. 显示目录内容 目录查看功能由 `fs_ls()` 实现。该函数遍历当前目录块中的所有目录项，并输出已经使用的目录项信息，包括文件类型、文件名、起始块号和文件大小。

输出格式如下：

```
1 [DIR ] dir1      first=3 size=0
2 [FILE] a.txt     first=4 size=22
```

3. 打开与关闭文件 为了记录已经打开的文件，本实验定义了打开文件表：

```
1 typedef struct {
2     int used;
3     int dir_block;
4     char filename[NAME_LEN];
5     char mode;
6 } OpenFile;
```

执行 `fs_open()` 时，系统根据文件名查找目录项，并在打开文件表中寻找空闲表项。如果使用读模式打开文件，则目标文件必须已经存在；如果使用写模式打开文件，而目标文件尚不存在，则系统会自动创建新的文件目录项。

文件打开成功后，函数返回文件描述符 `fd`。执行 `fs_close()` 时，只需要清空对应打开文件表项，即可释放该文件描述符。

4. 写入文件 文件写入功能由 `fs_write()` 实现。本实验采用覆盖写策略：如果目标文件原本已经占用数据块，则首先调用 `free_chain()` 释放原有 FAT 链，然后根据新写入数据的长度重新分配数据块。

写入过程中，系统会将数据按照每块 128 字节进行拆分，并通过 FAT 表将多个数据块连接起来。如果写入内容小于或等于 128 字节，则只需要分配一个数据块；如果写入内容大于 128 字节，则需要分配多个数据块，并形成 FAT 链。

5. 读取文件 文件读取功能由 `fs_read()` 实现。该函数首先根据文件描述符找到对应文件目录项，然后根据目录项中的 `first_block` 找到文件起始块。读取过程中，系统根据 FAT 表逐块向后遍历，直到读取完指定长度的数据，或者到达 FAT 链末尾。

6. 删除文件或目录 删除功能由 `fs_delete()` 实现。删除普通文件时，系统首先根据目录项中的 `first_block` 找到文件起始块，然后调用 `free_chain()` 释放文件占用的所有数据块，最后清空对应目录项。

对于目录，本实验只允许删除空目录。如果目标目录中仍然存在文件或子目录，则拒绝执行删除操作，从而避免目录结构出现不一致。

5.2.14 运行结果与分析

完成上述功能后，本实验依次对文件系统初始化、目录创建、文件写入、文件读取、文件删除以及跨块文件写入进行了测试。

1. 文件系统初始化 首先调用 `fs_format()` 对模拟磁盘进行格式化。运行结果显示，第 0、1、2 块均被标记为 `END`，第 3 块至第 19 块均被标记为 `FREE`。

```
1 format success
2 Block 0: END
3 Block 1: END
4 Block 2: END
5 Block 3: FREE
6 ...
7 Block 19: FREE
```

这说明超级块、FAT 表和根目录区已经被正确初始化，数据区处于空闲状态。

2. 创建目录 随后，在根目录中创建目录 `dir1`。运行结果如下：

```
1 mkdir success: dir1, block=3
2 [DIR ] dir1      first=3 size=0
3 Block 3: END
```

结果表明，系统成功为目录 `dir1` 分配了第 3 块，并在根目录中建立了对应目录项。FAT 表中第 3 块由 `FREE` 变为 `END`，说明目录创建过程正确。

3. 文件写入与读取 接下来，以写模式打开文件 `a.txt`，并向其中写入字符串 `Hello FAT File System!`。运行结果如下：

```
1 open success: a.txt, fd=0, mode=w
2 write success: fd=0, size=22 bytes
3 close success: fd=0
4 [FILE] a.txt      first=4 size=22
5 Block 4: END
```

由于写入内容长度为 22 字节，小于一个数据块大小，因此文件只占用第 4 块。随后，以读模式重新打开该文件，能够正确读取文件内容：

```
1 open success: a.txt, fd=0, mode=r
2 read success: fd=0, size=22 bytes
3 read result: Hello FAT File System!
4 close success: fd=0
```

这说明 `open`、`write`、`read` 和 `close` 功能均能够正常运行。

4. 文件删除与空间回收 删除文件 `a.txt` 后，运行结果如下：

```
1 delete file success: a.txt
2 Block 4: FREE
```

可以看到，原本由 `a.txt` 占用的第 4 块重新变为 `FREE`，同时根目录中已经不再显示该文件。这说明文件删除操作不仅清除了目录项，而且成功释放了文件占用的数据块。

5. 跨块文件写入 为了进一步验证 FAT 表的链式管理机制，本实验创建了文件 `big.txt`，并向其中写入 299 字节的数据。由于一个数据块只能保存 128 字节，因此该文件需要占用 3 个数据块。

运行结果如下：

```
1 open success: big.txt, fd=0, mode=w
2 write success: fd=0, size=299 bytes
3 close success: fd=0
4 [FILE] big.txt    first=4 size=299
5 Block  4: 5
6 Block  5: 6
7 Block  6: END
```

从 FAT 表可以看出，文件 `big.txt` 的数据块链接关系为：

Block 4 → Block 5 → Block 6 → END

随后，以读模式打开 `big.txt`，能够正确读取文件内容：

```
1 open success: big.txt, fd=0, mode=r
2 read success: fd=0, size=299 bytes
3 read big.txt first part:
4 ABCDEFGHIJKLMNOPQRSTUVWXYZABCDEFGHIJKLMNOPQRSTUVWXYZ...
5 close success: fd=0
```

该结果表明，本实验不仅能够正确读写单块文件，还能够为跨块文件建立 FAT 链，并沿 FAT 链完成文件内容读取。至此，任务一要求的目录创建、目录查看、文件删除、文件打开、文件读取、文件写入和文件关闭等功能均已实现。